

DHA Suffa University

Department of Computer Science



Deep Scan

P23F25

Final Year Project Report

Submitted by

Mir Balach Kamal (CS221213)
Abdul Wahab Imran (CS211178)
Ghulam Mustafa (CS221201)
Prince Laksh (CS211054)

Supervisor

Mr. Wajahat

In partial fulfilment of the requirements for the degree of
Bachelor of Science in Computer Science
2026

Certificate of Approval

It is certified that the work presented in this report, entitled Deep Scan was conducted by Mir Balach Kamal, Abdul Wahab Imran, Ghulam Mustafa and Prince Laksh under the supervision of Mr. Wajahat.

No part of this report has been submitted anywhere else for any other degree.

This report is submitted to the Department of Computer Science in partial fulfilment of the requirements for the degree of Bachelor of Science in Computer Science

at

DHA Suffa University (DSU)

Team Lead Name: _____ Mir Balach Kamal _____ Signature: _____

Member 1 Name: _____ Abdul Wahab Imran _____ Signature: _____

Member 2 Name: _____ Ghulam Mustafa _____ Signature: _____

Member 3 Name: _____ Laksh Prince _____ Signature: _____

Supervisor Name: _____ Mr. Wajahat _____ Signature: _____

Project Coordinator Name: ___ Mr. Hafiz Wahab Raza _ Signature: _____

Project Coordinator Name: ___ Mr. Farhan Siddiqui ___ Signature: _____

Author's Declaration

We declare that this project report was carried out in accordance with the rules and regulations of the DHA Suffa University (DSU). The work is original except where indicated by special references in the text and no part of the report has been submitted for any other degree. The report has not been presented to any other University for examination.

Dated:

Authors Signatures:

Mir Balach Kaml

Abdul Wahab Imran

Ghulam Mustafa

Laksh Prince

Plagiarism Undertaking

We, Mir Balach Kamal, Abdul Wahab Imran, Ghulam Mustafa, and Laksh Prince, solemnly declare that the work presented in the Final Year Project Report titled Deep Scan has been carried out solely by ourselves with no significant help from any other person except few of those which are duly acknowledged. I confirm that no portion of our report has been plagiarized and any material used in the report from other sources is properly referenced.

Dated:

Authors Signatures:

Mir Balach Kamal

Abdul Wahab Imran

Ghulam Mustafa

Laksh Prince

Acknowledgments

Firstly, We are thankful to Almighty Allah for helping us in completing our Final Year Project. We are thankful for the guidance, strength and health He gave us through out this journey.

We are thankful to our project supervisor for his guidance, support and advice through out this project, he helped us in my aspects from the start to the finish.

We are also thankful to the respected jury members who evaluated our project and gave us suggestions to add some features. Their suggested feature adding further value to our project.

Finally, A special thanks goes to our friends and family who gave their unconditional support and motivated us in the difficult times we faced in completing this project.

Document Information

Table 1: Document Information

Customer	DHA Suffa University (DSU)
Project Title	Deep Scan
Document	Final Year Project Report
Document Version	1.0
Identifier	P23F25-Final Report
Status	Final
Author(s)	All Members
Approver(s)	Mr. Wajahat
Issue Date	30/June/2026

Definition of Terms, Acronyms, and Abbreviations

Table 2: Definition of Terms, Acronyms, and Abbreviations

Term	Description
Deep Scan	AI-Powered Phishing Detection System
API	A way for different software applications to communicate with each other
URL	The web address of a website
SSL Certificate	A digital certificate of a website
Phishing	A cyberattack that uses fake websites or emails
FYP	Final Year Project
CORS	A browser security rule that controls data sharing between websites
Feature Extraction	The process of collecting important features from URLs and emails

Abstract

Phishing attacks are the most common cybersecurity attacks, attackers target users through malicious websites, QR codes and emails to steal their sensitive information. Mostly, current solutions only focus on one type of phishing attacks due to which users have to use different tools to scan URLs, emails and QR codes. We created Deep Scan, a system which allows to check URLs, emails and QR codes all in a single platform.

Our system uses Random Forest Algorithm for URL analysis and a DistilBERT model for email analysis. URLs in QR Codes are extracted through OpenCV and are checked through our URL analysis module. To improve security and the system also verifies SSL/TLS certificates.

Deep Scan is a web based platform where users can scan URLs, emails and QR codes, we also provide a chrome extension for real time analysis for URLs and emails, the Chrome Extension analyzes the emails opened in Gmail.

Overall, Deep Scan provides an easy and effective solution for protecting users from phishing attacks. It combines URL analysis, email analysis, and website certificate verification into a single platform, making it useful for improving online security.

Table of Contents

Certificate of Approval	2
Authors' Declaration	3
Acknowledgment	5
Document Information	6
Abstract	7
Chapter 1 INTRODUCTION	10
Description about Project	10
Details about the Domain	10
Relevant Background	10
Chapter 2 RELEVANT BACKGROUND & DEFINITIONS	11
Chapter 3 LITERATURE REVIEW & RELATED WORK	12
Literature Review	12
Related Work	12
Gap Analysis	12
Chapter 4 METHODOLOGY	13
Software Engineering Methodology	13
Project Methodology	Error! Bookmark not defined.
Chapter 5 EXPERIMENTAL EVALUATIONS & RESULTS	15
Evaluation Testbed	15
Results and Discussion	15
Chapter 6 CONCLUSION AND DISCUSSION	16
Limitations and Future Work	16
Reasons for Failure – If Any	16
REFERENCES	17
APPENDICES	18
A0. Copy of Project Registration Form	19
A1a. Project Proposal and Vision Document	20
A1b. Copy of Proposal Evaluation Comments by Jury	21
A2. Requirement Specifications	22
A3. Design Specifications	23
A4. Other Technical Detail Documents	24
Test Cases Document	24
UI/UX Detail Document	24
Coding Standards Document	24
Project Policy Document	24

User Manual Document	24
A5. Flyer & Poster Design	25
A6. Copy of Evaluation Comments	26
Copy of Evaluation Comments by Supervisor for Project – I Mid Semester Evaluation	26
Copy of Evaluation Comments by Supervisor for Project – I End Semester Evaluation	27
Copy of Evaluation Comments by Jury for Project – I End Semester Evaluation	28
Copy of Evaluation Comments by Supervisor for Project – II Mid Semester Evaluation	29
A7. Meetings' Minutes	32
A8. Document Change Record	33
A9. Project Progress (Gantt Chart)	34

CHAPTER 1

INTRODUCTION

Description of the Project

Our System Deep Scan is an AI Powered Phishing Detection System used to protect against phishing attacks that happen in the form of URLs, emails and QR codes.

Instead of using different tools for checking URLs, emails and QR codes, Deep Scan gives a single platform where users can check all these.

The system is developed using Python and Flask. The URL detection module uses a Random Forest machine learning model. It extracts different features from the URL, the email detection module analyzes email subject, sender information and email content. Email analysis module uses NLP with a DistilBERT model to understand the content of the email and identify whether the email is secure or suspicious.

Deep Scan also scans QR Codes. The system uses the OpenCV library to scan and decode QR codes. If a QR code contains a URL, then the extracted URL is checked through our URL model.

Another feature of our system is SSL/TLS certificate verification. This helps to check whether a website has a valid certificate or not.

Deep Scan is a web based platform, we have created a simple web application that even non technical users can use. To make the system more useful we have created a chrome extension that helps to verify URLs without going to the web app.

The chrome extension also works with Gmail. When a user opens a email in Gmail, the extension can collect the sender's email address, subject, message content, and links present in the email. This information is then analyzed through our Email analysis module and results are displayed to the user.

Details about the Domain

Deep Scan belongs to the field of Cybersecurity. Cybersecurity involves protecting computers, websites, networks, and digital information from cyber attacks. As more people are using online banking, social media, online shopping, and email services, protecting important information has become very important.

One of the most common type of cyber attack is phishing. In phishing attacks, attackers mimic to be trusted organizations or individuals to steal sensitive information and data. These attacks can lead to identity theft, financial loss, and unauthorized access to personal accounts.

Phishing attacks are performed in different forms. One of the most common method is through URLs, in URL attacks attackers create fake websites that look familiar to legitimate websites, in which users may unknowingly enter their details, which are then stolen by the attackers.

Another common method is email phishing, attackers send email that appear to have come from trusted organizations. In these emails, attackers create a sense of urgency by asking users to verify their accounts or update payment methods. If users think they are from trusted organizations and share their information they may share their sensitive information.

A newer type of phishing attack is QR code phishing, in these attacks harmful URLs are hidden inside QR codes. When user scan these QR codes they are redirected to fake websites.

Due to the increasing number of phishing attacks, there is a need of a system that detects all these three attacks in a single platform. Deep Scan addresses this problem by providing a platform where URLs and QR Codes can be through machine learning algorithms and Emails can be checked through NLP techniques.

Relevant Background

Phishing attacks have changed massively. At first, these attacks were simple and easy to intercept, but as the time passed the attackers developed more advanced phishing techniques. Attackers started creating convincing copies of legit websites, and bypassed traditional security measures.

The traditional measures used simple blacklists and rule-based detection, but these methods were not enough to stop the attacks, this led to the creation of detection systems that used machine learning, through which the system can identify new patterns and detect evolving attacks.

Current research in phishing detection focuses on different key approaches:

Feature-Based Analysis: This method looks at certain features from URLs and emails, like how long the domain name is, if there are any words that look strange and how the structure looks.

Content Analysis: This analyzes the content present on the actual website and emails. It uses NLP to find signs of phishing or something that might be a scam.

CHAPTER 2

RELEVANT BACKGROUND & DEFINITIONS

Phishing: Attack when attackers act like if they are some authentic organizations to steal important information.

Machine Learning: ML helps systems to learn and improve by finding out new patterns and learn from them.

Random Forest: It is an algorithm that combines decisions from multiple decision trees to make more accurate predictions.

Natural Language Processing (NLP): NLP helps systems to understand and interpret emotions inside the message.

Feature Extraction: Processes of extracting useful features through which the system can make accurate predictions.

URL (Uniform Resource Locator): URL tells your browser where to go and how to access something online.

API (Application Programming Interface): A set of protocols and tools that allows one application to interact with another.

Chrome Extension: This is a program to add functionality to Chrome browser.

Cross-Origin Resource Sharing (CORS): It is a security rule in browsers which decides if a webpage can request data from another web page.

SSL Certificate: It shows if the website is secure and authentic.

False Positive: It happens when the system detects something as dangerous but it is safe.

False Negative: It happens when the system detects something as safe but it is dangerous.

CHAPTER 3

LITERATURE REVIEW & RELATED WORK

Literature Review

Research in phishing detection has transformed through several phases, in each phase they solved the problems in the existing approaches. In early phases they focused on simple blacklist based systems, but then the researchers realized that there is need of more advance detection methods. In 2014 Mohammad did detail research on URL phishing detection with the use of machine learning. During this research it was found out that by extracting features from URLs can massively improve the detection capabilities in comparison with the existing blacklist approaches. Recently researchers have tried using deep learning in phishing detection, this showed great results in detecting attacks that existing approaches were not able to.

Researchers studied different ML algorithms for phishing detection such as Random Forest, Decision Tree, SVM, etc. These algorithms can learn from previous data and find patterns that help to classify URLs. Among these algorithms, Random Forest showed good accuracy and reliable performance.

Another important area in phishing detection is email detection. Researchers have tried different methods used in NLP, like analyzing emotions in text, finding important words and detecting language patterns to spot scam emails.

Researchers have also explored website security features to improve phishing detection. One of these features is SSL certificate, this certificate helps to verify the authenticity of a website. However, researchers have also found that phishing websites can also use valid SSL certificates, therefore SSL certificate checking should be used as an additional feature combined with the detection techniques instead of being used alone

Deep Scan is based on these research developments, our system uses machine learning for URL detection, NLP for email analysis and SSL certificate analysis in a single platform. Our system also includes a web interface and a chrome extension to provide real time protection.

Related Work

Many phishing detection system have been developed. Google safe browsing detects phishing websites by comparing URLs with a database of known websites. It is fast and reliable for detecting reported websites, but it cannot always identify newly created websites that are not in the database yet.

Virus Total also allows users to scan URLs and provide detailed reports but it is a paid tool. Our system addresses critical gaps in existing solutions by giving free protection against both URL and emails phishing attacks.

Gap Analysis

Even with the huge progress made in phishing detection research there are some gaps which remain even today:

Limited Real-time Performance: Many systems take too long to scan data, this makes them unsuitable in situation in which users need fast accurate response.

High False Positive Rates: Many systems sometimes flag safe URLs and emails by mistake which frustrates the users and which leads to the users ignoring real threats.

Inadequate Email Analysis: URL detection has made progress but email analysis has been left lagging behind, the systems still look for common keywords that are used by attackers rather than analyzing the content and emotions present in the email.

Lack of Integrated Solutions: Many systems provide URL, QR or email detections but very few provide all at the same place.

CHAPTER 4

METHODOLOGY

Software Engineering Methodology

We built our system using agile methodology, in which we developed and tested the system in cycles, by using this methodology it helped us to improve the system and fix the bugs.

Requirements Gathering: At first, we analyzed the user needs and the existing solutions which helped us defined the requirements of our system

System Design: We built the system in parts, like URL detection, email analysis, API development, user interface because it helped us to complete one module at a time and test each part.

Implementation Strategy: We developed the system in steps, first we completed the URL detection, then email detection, then we created the API, then we created user interface, and then created a chrome extension to provide real time protection to the users.

Testing Approach: We tested each module after development to make sure that the system is reliable and working.

Quality Assurance: We monitored our system performance to make sure the system is reliable.

Project Methodology

Agile Methodology

Sprint 1 - Research and Planning: First we gathered the necessary data and started testing different AI models to find which one is good.

Sprint 2 - Algorithm Development: We used PhiUSIIL dataset for URL detection and CEAS_08 dataset for email analysis, we trained and built our system using these datasets which we downloaded from Kaggle.

Sprint 3 - System Integration: We connected all the separate modules in a single system. We wanted to make sure that system worked correctly.

Sprint 4 - User Interface Development: We built a friendly user interface where user can check URLs and emails and created chrome extension to provide real time protection.

Sprint 5 - Testing and Validation: We tested the system with both training data and real- world examples to make sure the system was reliable and worked correctly.

Sprint 6 - Documentation: Lastly, we made the documentation about our system.

Chapter 5

EXPERIMENTAL EVALUATIONS & RESULTS

Evaluation Testbed

We tested our system with training data as well as the real world examples to make sure that our system was reliable.

Dataset Selection: We used two datasets for training and evaluation, the PhiUSIIL phishing URL dataset which had more than 200,000 URLs, this helped us in extracting important URL features for the training of URL detection model. We used CEAS_08 dataset for email detection, this dataset had more than 30,000 emails which helped us to train and test email analysis model.

Results and Discussion

The testing showed that overall our system is good at catching malicious URLs and email, and it rarely flagged legit websites by mistake.

URL Detection Performance: Our Random Forest model achieved approximately 96% accuracy on the test dataset. The system was fast in detecting URLs.

Email Detection Performance: Our email analysis module classified test emails correctly. It also gave great accuracy in detecting phishing emails

System Integration Performance: When we combined all the modules together and tested we found out that the complete system ran smoothly.

Comparative Analysis: When we compared our system to existing solution our system showed good competitiveness.

CHAPTER 6

CONCLUSION AND DISCUSSION

REFERENCES

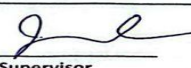
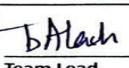
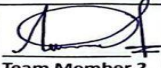
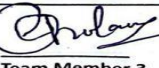
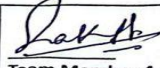
1. Kalla, D., & Kuraku, S. (2023). *Phishing website URL 's detection using NLP and machine learning techniques*. Journal on Artificial Intelligence, 5(1), 145–162.
<https://doi.org/10.32604/jai.2023.043366>
2. Lingaiah, G., & Pandey, R. K. (2023). *Comparative performance study of machine learning algorithms for phishing URL detection in cyber security systems*. Journal of Computational Analysis and Applications, 31(4), 1865–1873.
3. Sahingoz, O. K., Buber, E., Demir, O., & Diri, B. (2019). Machine learning based phishing detection from URLs. Expert Systems with Applications , 117, 345- 357.
<https://doi.org/10.1016/j.eswa.2018.09.029>
4. Tsai, E., Kumar, D., Raman, R. S., Li, G., Eiger, Y., & Ensafi, R. (2023). *CERTainty: Detecting DNS manipulation at scale using TLS/SSL certificates*. Proceedings on Privacy Enhancing Technologies, 2023(3), 122–137. <https://doi.org/10.56553/popets-2023-0073>

APPENDICES

List of Appendices

- A0. Copy of Project Registration Form
- A1a. Project Proposal and Vision Document
- A1b. Copy of Proposal Evaluation Comments by Jury
- A2. Requirement Specifications
- A3. Design Specifications
- A4. Other Technical Details
- Test cases
- UI/UX Details
- Coding Standards
- Project Policy
- A5. Flyer & Poster Design
- A6. Copy of Evaluation Comments
 - Copy of Evaluation Comments by Supervisor for Project – I Mid Semester Evaluation
 - Copy of Evaluation Comments by Supervisor for Project – I End Semester Evaluation
 - Copy of Evaluation Comments by Jury for Project – I End Semester Evaluation
 - Copy of Evaluation Comments by Supervisor for Project – II Mid Semester Evaluation
 - Copy of Evaluation Comments by Jury for Project – II Mid Semester Evaluation
- A7. Meetings' Minutes
- A8. Research Paper if any

A0. COPY OF PROJECT REGISTRATION FORM

FYP-I Registration Form FALL 2025				
Project Type* (please tick one)	☐ Research Track	☐ Service Track	☒ Product Track	*to be selected by supervisor
Idea proposed by (please tick one)	☐ Supervisor	☒ Students	☐ Client	
Project Title: + Descriptive Title	Deep Scan. AI-powered Phishing Detection System.			
Supervisor Name:	Zongheng Zhai		Note: A faculty member <u>cannot</u> supervise more than 5 projects accumulating both FYP-1 and FYP-II at a time.	
Co-Supervisor(s) Name:	HAMMAD KHAWAR		Hamad 03/12/2025	
Client Name	Optional (in case of service track)		(Attached NDA / Letter)	
Team Lead Name:	Mix Balach kamal	Reg. No.	CS-221213	
Team Member 2 Name:	ABDUL WAHAB	Reg. No.	CS-211178	
Team Member 3 Name:	Ghulam Mustafa	Reg. No.	CS-221201	
Team Member 4 Name:	Prince Laksh	Reg. No.	CS-211054	
Project Keywords (Comma separated terms describing your project domain)				
NLP and Machine Learning.				
Project Abstract				
The Project aims to develop intelligent system that can detect detect phishing emails or fake login pages using machine learning and natural languages. Processing (NLP) techniques.				
 Supervisor	 Team Lead	 Team Member 2	 Team Member 3	 Team Member 4

A1A. PROJECT PROPOSAL AND VISION DOCUMENT

1 Introduction

We are making a system that helps to protect users from phishing attacks, attackers attack users by sending them URL's, QR codes and emails that look real and legitimate but they trick users into giving them important credentials. Phishing detection system will help users by checking both URLs and emails in one place. We will build the system in two parts. In the part 1 we will use machine learning to check URLs and rule based analysis to check emails. In the part 2 we will add a feature that checks the certificates of websites as it will help in making prediction, use advance NLP techniques to analyze emails and also create a Chrome extension that will protect users in real time.

1.1 Problem Statement

The big problem is that phishing is getting worse. The ways we protect ourselves are not changing. Most tools only check one thing, like a link or an email. They do not check both at the same place. This makes it easy for hackers to trick people. Also many tools are too slow or too hard to use. Phishing detection system will fix this by being fast easy to use and checking both URLs, QR codes and scam emails in one place. We need a system that's easy to use and protects us in real time.

1.2 Project Motivation

We created Deep Scan because phishing is a huge problem, many people are targeted daily because of these attempts. The main issue is that most security tools do not check both links and emails at the time. This makes it easy for hackers to trick people. We saw how hard it is for people to spot these threats so we decided to create a system that's accurate and easy to use. Because of these challenges we will build the system in two parts. In the part 1 we will get the URL protection working and create a user-friendly web interface. In the part 2 we will use advance NLP techniques to detect emails and create a Chrome extension to protect users in real time.

1.3 Objectives

- To develop a web-based platform that allows users to scan URLs, emails and QR codes from a single interface.
- To develop a URL detection module that extracts different features to classify URLs as legitimate or Phishing.
- To develop an email analysis module to detect emails based on their content.
- To integrate QR code analysis by extracting URLs from QR codes and analyze them through our URL detection module.
- To improve phishing detection by checking websites certificates.
- To develop a chrome extension that makes it possible to analyze URLs and Gmail emails directly from their browser.

1.4 Literature Review

Most existing work on phishing focuses on just one thing at a time. Some researchers use Random Forest or SVM to look at URL features like how long the link is or if it has weird symbols. Others use NLP to look for suspicious words in emails. The problem is that these are almost always separate studies or separate tools. There isn't much out there that puts both together in a way that a regular person can actually use.

2 Project Vision

2.1 Business Case and SWOT Analysis

Deep Scan is built to solve a problem. Most high-end security tools are too pricey and complex for people or small teams. Our goal is to create a platform. It brings URL and email phishing detection together in one single web dashboard. The project aims to stop phishing attacks. We want to make safety easy for everyone by offering an interface for checking links and emails. Deep Scan helps people identify phishing attacks. It makes digital safety reachable, for all.

SWOT Analysis

S. No	Strengths	Weaknesses	Opportunities	Threats
1	Covers both URLs, QR codes, and Emails in one single interface.	Accuracy depends heavily on the quality of training data.	Increasing demand for easy-to-use tools for non-tech users.	Hackers constantly change their tactics to bypass ML models.
2	The development plan allows us to add more advanced models later.	As a new tool, it takes time to build user trust compared to Google or Norton.	Moving into the browser market offers a huge user base.	Tech giants like Google are constantly improving their own built-in filters.
3	Built using open-source tools (Python, Flask), making it accessible.	Current system works for English only	Releasing the platform for free to promote cybersecurity awareness and provide accessible protection for all users	Misidentifying a safe site as a threat could frustrate users.

2.2 Business Objectives and Success Criteria

- **Core Functionality:** Our goal is to build and launch a system that correctly detects URLs, QR codes and emails. This system needs to be able to handle URLs and emails. We will know it is a success if it can correctly identify phishing samples and give us results.
- **Ease of Use:** We want to create a web dashboard that anyone can use. We do not want people to need a manual to use it. If half of our non technical users are able to scan and understand the results in their first try we will consider this a success.
- **System Performance:** We need to make sure that the system is fast, so users do not have to wait for their results.

2.3 Project Risks and Risk Mitigation Plan

We have a few risks to think about.

1. **Software Dependency Risks:** If libraries like Flask or Scikit-learn update it could cause problems with our code. This means we will have to spend time fixing it.
2. **System Performance:** When we add complex machine learning models later the system might get slower.
3. **Time Constraint Risk:** We might not have time to finish both phases of the project.
4. **Resource Limitation Risk:** We might not have computer power to train our machine learning models or host the system.

Risk Mitigation Plan:

1. **For Software Updates:** We will use versions of every library. We will keep a list of what we need. This way if a new version of Flask comes out our project will still work.
2. **For System Performance:** We will keep our code simple. We will use models that are fast and do not need a lot of power to run.
3. **For Time Constraints:** We will split the work into two phases. We will complete different modules step by step.
4. **For Hardware:** If our computers are too slow to train the models we will use tools to do the heavy work, for the machine learning models.

2.4 Assumptions and Dependencies

Assumptions:

1. **User Access:** We assume that users have an internet connection and they know how to use a web browser to check links and URLs.
2. **URL Format:** We expect the links that users send to be in the format like the kind that starts with HTTP or HTTPS.
3. **Language:** We assume that all the email content will be written in English because our system will only be analyzing text submitted in English.

Dependencies:

1. **Technical Framework:** The project relies on Flask to make the web server work and it uses scikit-learn and pandas to handle the machine learning part.
2. **Data Availability:** The URL model is only accurate if we have the datasets to train it so we really need to have access, to that data.

3 Project Scope

3.1 In Scope

The scope of this project is to develop a website with separate tabs for URL, QR codes and email analysis, where users can submit URLs, emails and QR codes for analysis. For URL analysis we will be using Random Forest algorithm. Email analysis module uses NLP with a DistilBERT model to understand the content of the email and identify whether the email is secure or suspicious. The system uses the OpenCV library to scan and decode QR codes. If a QR code contains a URL, then the extracted URL will be checked through our URL model. We will also check certificates of the websites. Building a chrome extension for both URL and email analysis is part of the scope, we will make a chrome extension for Gmail, through which users can open their Gmail inbox and scan the emails they receive.

3.2 Out of Scope

There are some things that we will not be part of our project, we will not supporting bulk URLs scanning, this means one URL can be scanned at a time. Our project will not require any registration or login, it can be used without making any account. Our project is a web-based platform so we will not be making any mobile application. The system will only analyze emails written in English language.

4 Proposed Methodology

4.1 SDLC Approach (Waterfall/Agile/any model)

The Project is divided into two separate phases FYP1 and FYP2 and we have adopted Agile Methodology

4.2 Team Role & Responsibilities

Mr. Wajahat - Project Supervisor

Mr. Hammad Khawar - Project Co-supervisor

Mir Balach - Team Leader

Responsibilities – Backend, Model Training, Documentation

Ghulam Mustafa - Team Member

Responsibilities – Backend, Documentation

Abdul Wahab Imran– Team Member

Responsibilities – Frontend, Documentation

Laksh Prince - Team Member

Responsibilities – Frontend, Documentation

4.3 Requirement Development

We got our training data from Kaggle, the PhiUSIIL dataset. This dataset has features that help detect malicious URLs and for email analysis we used CEAS_08 dataset.

- **Analysis and Design:** We chose to check both URLs and emails because looking at one is not enough for full protection. For URLs we used a Random Forest model. For emails we used NLP approach. Our system has three parts:
- **Presentation Layer:** A web dashboard with tabs for users to interact with.
- **Application Layer:** API that runs our detection logic and models.
- **Data Layer:** We store our datasets and trained models in files because it easy to use.

4.4 High-level Architecture / Design

Presentation Tier (UI): This is the part users see. It's a web dashboard with tabs to paste links or email text. The UI sends the information to the backend. Shows the results, including a threat level and a score that shows how confident the system is.

Application Tier (Logic): This is like the brain of the project. It connects users to our detection models. We have a URL engine that uses Random Forest and an email engine that uses NLP tools like NLTK. We used a Flask API to manage requests handle errors and make sure data is processed correctly before sending results.

Data Tier (Storage): This is where the systems data is stored. It keeps CSV files for training, pickle files, for our trained model and email detection rules. The application layer uses this tier to load models when users submit a scan.

5 Project Planning

5.1 Gantt Chart

	A	B	C	E	F	G	H	I
1	[P23F25] Deep Scan							
2	FYP - Fall 25							
3								
4	Project Start Date: 11/30/2025 (Sunday) y Week 7							
5	Team Members: Mir Balach Kai Ghulam Mustafa, Laksh P Abdul Wahab In							
6								
7	WBS	TASK	Assigned To	START	DAYS	END	% DONE	WORK DAYS
8	1	FYP-I				-		-
9	.1	Requirements Analysis & All Members		Sun 11/30/25	14	Sat 12/13/25	100%	10
10	.1	Data Collection & Cleanir	Mir Balach	Sun 12/14/25	21	Sat 1/03/26	100%	15
11	.1	URL Model Training	Mir Balach	Sun 12/14/25	28	Sat 1/10/26	100%	20
12	.1	Email Rule-based Engine	Ghulam Mustafa	Sun 1/04/26	21	Sat 1/24/26	100%	15
13	.1	Frontend	Laksh & Wahab	Sun 1/04/26	14	Sat 1/17/26	100%	10
14	.1	Flask API Integration	Balach & Musta	Sun 1/11/26	15	Sun 1/25/26	100%	10
15	.1	Initial Testing & Documentation	All Members	Mon 1/12/26	8	Mon 1/19/26	100%	6
16	1	FYP-II				-		-
17	.1	Research Advanced ML	Mir Balach	Sun 2/22/26	21	Sat 3/14/26	100%	15
18	.1	Chrome Extension Devel	Laksh & Wahab	Sun 2/22/26	35	Sat 3/28/26	100%	25
19	.1	Advance Email Analysis i	Balach & Musta	Sun 3/29/26	28	Sat 4/25/26	100%	20
20	.1	System Integration and h	Ghulam Mustafa	Sun 4/26/26	21	Sat 5/16/26	100%	15
21	.1	Performance Tuning & Fi	All Members	Sun 5/17/26	14	Sat 5/30/26	100%	10
22	.1	Final Documentation	All Members	Sun 5/31/26	21	Sat 6/20/26	100%	15
23	.1	Final Presentation Prep & All Members		Sun 6/21/26	21	Sat 7/11/26	100%	15

6 Project Requirements

6.1 Software tools requirements

We used PyCharm for development and our project runs on Windows 10/11. Programming & Frameworks:

- Backend: We used Python with the Flask for the API.
- Frontend: We use HTML, CSS and JavaScript for user interface.
- Machine Learning & NLP: We used scikit-learn for our Random Forest model and pandas/numpy to handle our datasets. For QR Codes analysis we used OpenCV library. For email analysis we use NLTK and TextBlob to process text and detect patterns.

Testing & API Tools:

- Postman: We test our API endpoints with Postman to ensure they return the data.
- Flask-CORS: This allows our frontend to communicate with our backend without security blocks.
- Pytest: We run basic unit tests on our code with Pytest.

6.2 Hardware requirements

Minimum Requirements:

A basic laptop is enough for basic development and running of the system.

A stable internet connection is needed to download the Python libraries.

7 Budget/Costing

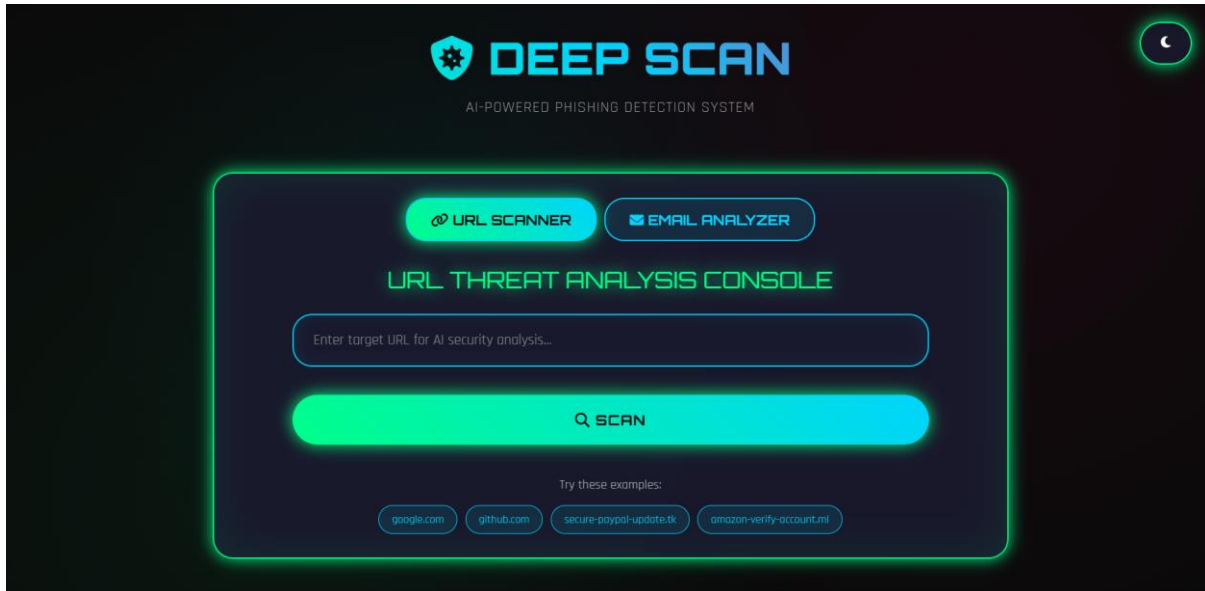
7.1 Estimated Budgeted Cost - of the Project

- Total Hours - 200_ hrs
- Per Hour Rate - N/A_ Rs
- Total Hours Rate - _N/A Rs
- Hardware Cost - _N/A Rs
- Other resources - N/A_ Rs
- Total cost Rs. - _N/A Rs

8 Project Deliverables

1. **Requirements and Design Docs** We are providing a set of documents that define what our project does. These documents include user requirements for URL and email scanning. They also have system diagrams that show how data flows through our detection engines.
2. **The Deep Scan Application:** We are providing a working version of our platform. In Phase 1 this is a web-based dashboard. Users can manually submit links and text on this dashboard. In Phase 2 this deliverable expands to include the Chrome browser extension. This extension allows for real-time protection while surfing.
3. **Source. Datasets:** We are providing the organized codebase. This includes the Flask backend frontend files and Chrome extension scripts. We are also providing CSV datasets. Saved pickle files for the models. A simple README and requirements file are included for setup.
4. **Reports:** We are providing a report in both hard and soft copies. This report details our methodology, results and performance metrics. It shows how our system has evolved through both phases.

9 Proposed GUI (Disposable Prototype)



10 References

1. Kalla, D., & Kuraku, S. (2023). *Phishing website URL's detection using NLP and machine learning techniques*. Journal on Artificial Intelligence, 5(1), 145–162.
<https://doi.org/10.32604/jai.2023.043366>
2. Lingaiah, G., & Pandey, R. K. (2023). *Comparative performance study of machine learning algorithms for phishing URL detection in cyber security systems*. Journal of Computational Analysis and Applications, 31(4), 1865–1873.
3. Sahingoz, O. K., Buber, E., Demir, O., & Diri, B. (2019). Machine learning based phishing detection from URLs. Expert Systems with Applications , 117, 345- 357.
<https://doi.org/10.1016/j.eswa.2018.09.029>
4. Tsai, E., Kumar, D., Raman, R. S., Li, G., Eiger, Y., & Ensafi, R. (2023). *CERTainty: Detecting DNS manipulation at scale using TLS/SSL certificates*. Proceedings on Privacy Enhancing Technologies, 2023(3), 122–137. <https://doi.org/10.56553/popets-2023-0073>

A1B. COPY OF PROPOSAL EVALUATION COMMENTS BY JURY

P23F25 - Deep Scan	Approved
P23F25 - Deep Scan	Approved
P23F25 - Deep Scan	The idea seems too basic considering Google and existing solutions are better.

A2. REQUIREMENT SPECIFICATIONS

1. Introduction

We are making a system which gives a platform to scan URLs, emails and QR Codes at a single platform. Attackers attack users by sending them URL's and emails that look real and legitimate but they trick users into giving them important credentials. We will build the system in two parts. In FYP-1 we will use machine learning to check URLs and natural language processing to check emails. In the FYP-2 we will add a feature that checks the certificates of websites as it will help in making prediction, we will also add QR codes detection and also create a Chrome extension that will protect users in real time and make our email detection better.

1.1. Purpose of Document

The purpose of this document is to help explain what our system does and what features it has. This document will work as a guide as it explains how the system works and how the system will be made in two parts.

1.2. Intended Audience

Intended audience are the jury evaluators, FYP coordinators, our project supervisors and the developers involved in making Deep Scan.

2. Overall System Description

2.1. Project Background

Phishing attacks have changed massively. At first, these attacks were simple and easy to intercept, but as the time passed the attackers developed more advanced phishing techniques. Attackers started created convincing copies of legit websites, and bypassed traditional security measures.

The traditional measures used simple blacklists and rule-based detection, but these methods were not enough to stop the attacks, we as a team want to add contribution in cyber security by making a system that can check both URLs and emails at a same place.

2.2. Project Scope

In Scope

The scope of this project is to develop a website with separate tabs for URL, QR codes and email analysis, where users can submit URLs, emails and QR codes for analysis. For URL analysis we will be using Random Forest algorithm. Email analysis module uses NLP with a DistilBERT model to understand the content of the email and identify whether the email is secure or suspicious. The system uses the OpenCV library to scan and decode QR codes. If a QR code contains a URL, then the extracted URL will be checked through our URL model. We will also check certificates of the websites. Building a chrome extension for both URL and email analysis is part of the scope, we will make a chrome extension for Gmail, through which users can open their Gmail inbox and scan the emails they receive.

Out of Scope

There are some things that we will not be part of our project, we will not supporting bulk URLs scanning, this means one URL can be scanned at a time. Our project will not require any registration or login, it can be used without making any account. Our project is a web-based platform so we will not be making any mobile application. The system will only analyze emails written in English language.

2.3 Project Objectives

- To develop a web-based platform that allows users to scan URLs, emails and QR codes from a single interface.
- To develop a URL detection module that extracts different features to classify URLs as legitimate or Phishing.
- To develop an email analysis module to detect emails based on their content.
- To integrate QR code analysis by extracting URLs from QR codes and analyze them through our URL detection module.
- To improve phishing detection by checking websites certificates.
- To develop a chrome extension that makes it possible to analyze URLs and Gmail emails directly from their browser.

2.4 Stakeholders

The stakeholders are the jury evaluators, our FYP coordinators, our project supervisors and the team that developed the system.

2.5 Operating Environment

Our system will be a web based platform where both URLs and emails can be checked using both desktops and laptops. We will make the frontend of our system using HTML, CSS and JSS, while backend will be made using Flask and Python. Stable internet connection is required to use our system.

2.6 System Constraints

Our system has some constraints like it only handles HTTP/HTTPS protocols, and emails submitted in English language can be analyzed and it may only be used if the user has stable internet connection.

2.7 Assumptions & Dependencies

We have made some assumptions and dependencies for our project:

Assumptions:

- **User Access:** We assume that users have an internet connection and they know how to use a web browser to check links and URLs.
- **URL Format:** We expect the links that users send to be in the format like the kind that starts with HTTP or HTTPS.
- **Language:** We assume that all the email content will be written in English because our system will only be analyzing text submitted in English.

Dependencies:

- **Technical Framework:** The project relies on Flask to make the web server work and it uses scikit-learn and pandas to handle the machine learning part.
- **Data Availability:** The URL model is only accurate if we have the datasets to train it so we really need to have access, to that data.

3 External Interface Requirements

3.1 Hardware Interfaces

The system is a simple web based platform so it does not need any extraordinary hardware to work, it can run on a basic computer.

3.2 Software Interfaces

Our system is built using Python and we used Flask for the backend. Scikit learn is used to detect phishing URLs, for email analysis we used Natural Language toolkit and TextBlob. For QR codes analysis we have used OpenCV library. Our frontend is made using HTML, CSS, and JSS.

3.3 Communications Interfaces

The system uses HTTP/HTTPS protocols and the backend runs on port 5000. It has CORS which enables browsers to access it.

4 Functional Requirements

4.1 Functional Hierarchy

Ref #	Functions	Category	Attribute	Details & Boundary Constraints
1	Phishing URL analysis	Evident	Detection	Users can submit URLs for threat detection.
2	Phishing email analysis	Evident	Detection	Users can submit email content for threat detection.
3	Threat classification display	Evident	User interface	System displays results
4	Dual mode interface	Evident	User Interface	Single web interface with ease of navigation in between URL and email analyzer
5	API access	Evident	Integration	Other programs can use the system through API calls with JSON responses
6	Process URL features automatically	Hidden	Processing	System automatically analyzes different URL characteristics
7	Process email text	Hidden	Processing	System automatically analyzes email text using language processing tools
8	Chrome Extension	Hidden	Security	Real-time URL monitoring and automatic phishing alerts during web browsing

4.2 Use Cases

4.2.1 Use Case 1

Analyze URL

Section	Main
Name:	Analyze URL
Actors:	End User, System Interface
Description:	Purpose is to facilitate the detection of phishing URLs through analysis. A user submits a suspicious URL for analysis. The system extracts features and applies detection algorithm to classify the URL. Once analyzed, the user receives result. The system provides real-time protection through browser extension.
Cross References:	Functions: 1,3,4,5,6,8
Pre-Conditions	System is operational and user has internet connectivity
Successful Post Conditions	User receives phishing classification with confidence score.
Failure Post Conditions	System displays error message

4.2.2 Use Case 2

Analyze Email

Section	Main
Name:	Analyze Email
Actors:	End User, System Interface
Description:	Purpose is to facilitate the detection of phishing emails through analysis. A user submits suspicious email for analysis. The system processes the email text using NLP techniques. Once analyzed, the user receives the result.
Cross References:	Functions: 2,3,4,5,7
Pre-Conditions	System is operational and user has internet connectivity
Successful Post Conditions	User receives phishing classification with confidence score.
Failure Post Conditions	System displays error message

5 Non-functional Requirements

5.1 Performance Requirements

Our system should be fast and accurate when analyzing URLs and emails.

5.2 Safety Requirements

Our system should check the content submitted and make predictions that clear and direct so that the users are not confused about whether a link or URL is dangerous.

5.3 Security Requirements

Our platform should be easy to use so that even non technical users can easily navigate and do analysis.

5.4 User Documentation

Our project will include documentation, This will include instructions on how to scan URLs and emails.

1. Kalla, D., & Kuraku, S. (2023). *Phishing website URL's detection using NLP and machine learning techniques*. Journal on Artificial Intelligence, 5(1), 145–162.
<https://doi.org/10.32604/jai.2023.043366>
2. Lingaiah, G., & Pandey, R. K. (2023). *Comparative performance study of machine learning algorithms for phishing URL detection in cyber security systems*. Journal of Computational Analysis and Applications, 31(4), 1865–1873.
3. Sahingoz, O. K., Buber, E., Demir, O., & Diri, B. (2019). Machine learning based phishing detection from URLs. Expert Systems with Applications , 117, 345- 357.
<https://doi.org/10.1016/j.eswa.2018.09.029>
4. Tsai, E., Kumar, D., Raman, R. S., Li, G., Eiger, Y., & Ensafi, R. (2023). *CERTainty: Detecting DNS manipulation at scale using TLS/SSL certificates*. Proceedings on Privacy Enhancing Technologies, 2023(3), 122–137. <https://doi.org/10.56553/popets-2023-0073>

A3. DESIGN SPECIFICATIONS

1 Introduction

1.1 Purpose of Document

The purpose of this document is to explain how we plan to build our system Deep Scan. This document tells the step by step development plan that we are going to use.

1.2 Intended Audience

Intended audience are the jury evaluators, FYP coordinators, our project supervisors and the developers involved in making Deep Scan.

1.3 Project Overview

We want to create a system that helps to protect from phishing URLs, QR Codes and emails, we will be building our project in two stages. In first stage we will create a user interface, use machine learning algorithm for URL detection and simple rule based analysis for email analysis. In the second stage, we will use advance natural processing language techniques to provide better email analysis, also analyze certificates of websites and make a chrome extension for real time protection.

1.4 Scope

In Scope

The scope of this project is to develop a website with separate tabs for URL, QR codes and email analysis, where users can submit URLs, emails and QR codes for analysis. For URL analysis we will be using Random Forest algorithm. Email analysis module uses NLP with a DistilBERT model to understand the content of the email and identify whether the email is secure or suspicious. The system uses the OpenCV library to scan and decode QR codes. If a QR code contains a URL, then the extracted URL will be checked through our URL model. We will also check certificates of the websites. Building a chrome extension for both URL and email analysis is part of the scope, we will make a chrome extension for Gmail, through which users can open their Gmail inbox and scan the emails they receive.

Out of Scope

There are some things that we will not be part of our project, we will not supporting bulk URLs scanning, this means one URL can be scanned at a time. Our project will not require any registration or login, it can be used without making any account. Our project is a web-based platform so we will not be making any mobile application. The system will only analyze emails written in English language.

2 Design Considerations

2.1 Assumptions and Dependencies

We have made some assumptions and dependencies for our project:

Assumptions:

- **User Access:** We think that users have an internet connection and they know how to use a web browser to check links and URLs.
- **URL Format:** We expect the links that users send to be in the format like the kind that starts with HTTP or HTTPS.
- **Language:** We assume that all the email content will be written in English because our system will only be analyzing text submitted in English.

Dependencies:

- **Technical Framework:** The project relies on Flask to make the web server work and it uses scikit-learn and pandas to handle the machine learning part.
- **Data Availability:** The URL model is only accurate if we have the datasets to train it so we really need to have access, to that data.

2.2 Risks and Volatile Areas

Risks:

We might not have the time to finish all the tasks in time, because we have limited development time.

Volatile Areas:

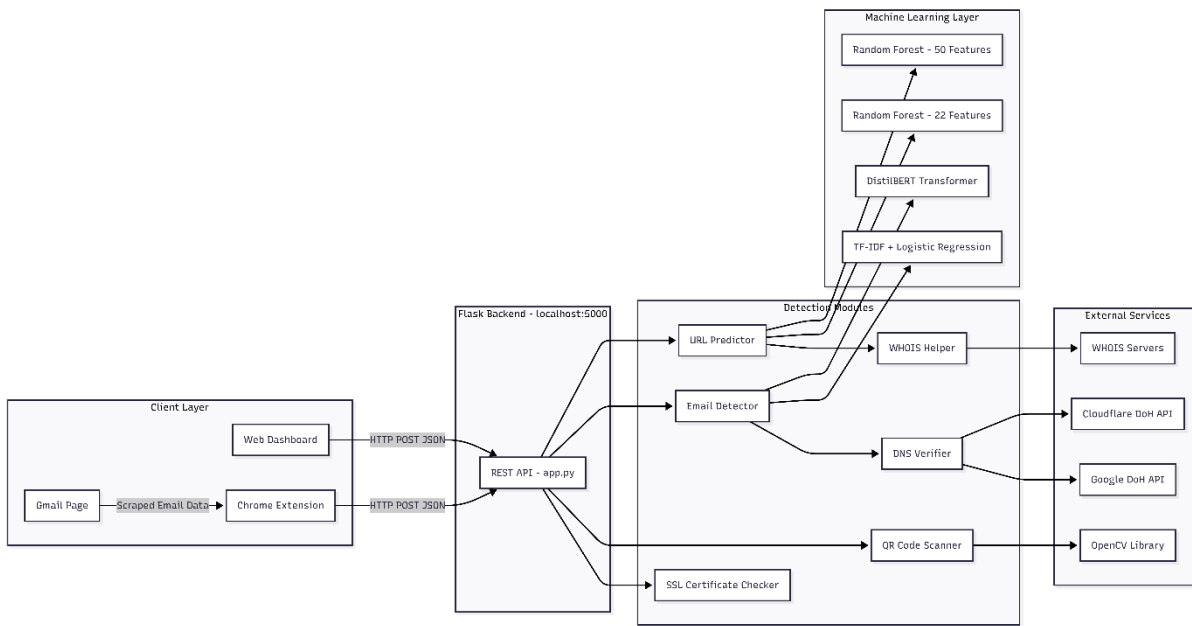
We will have to retrain our model when new phishing techniques are discovered.

We will have to update our email analysis model as scammers invent new techniques.

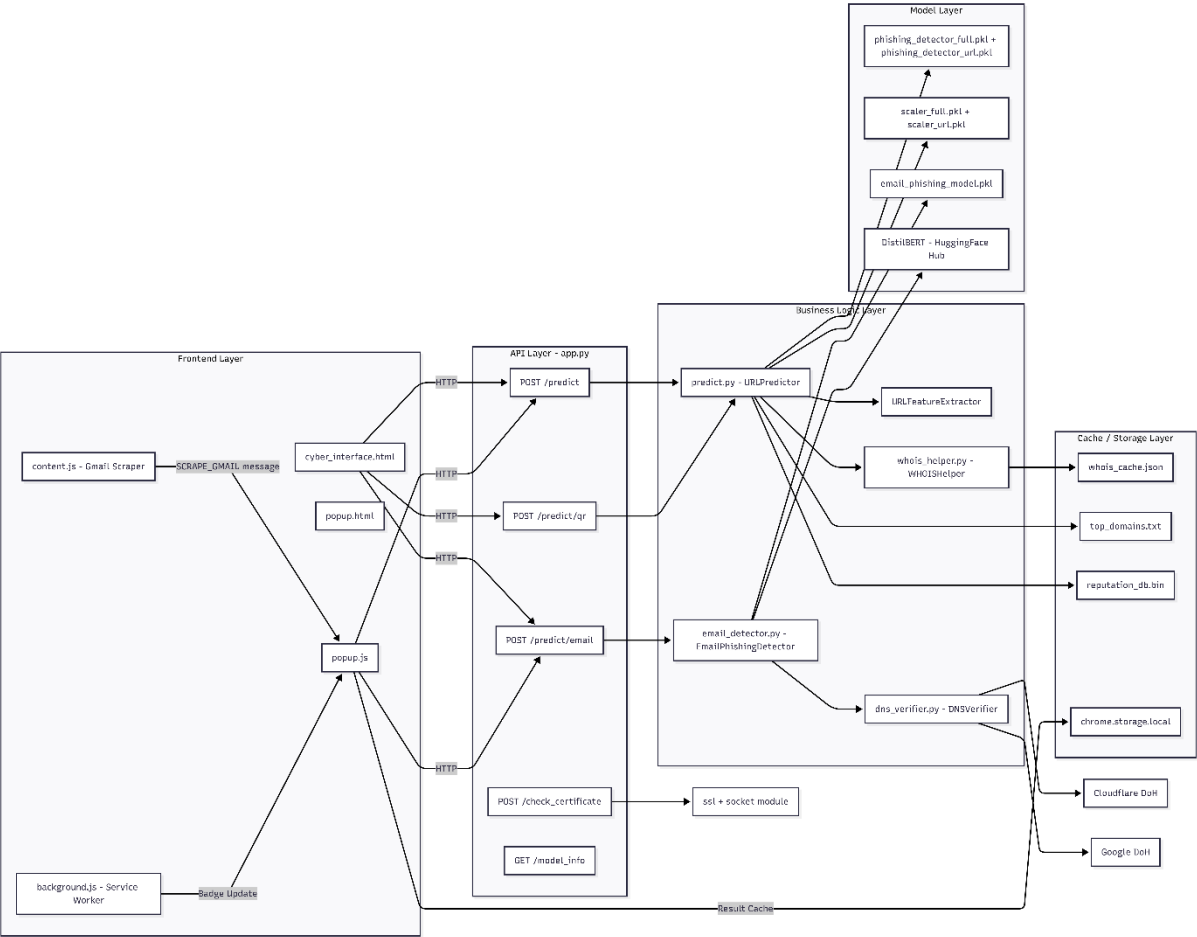
We might have to change our web interface based on what users think and results of our testing.

3 System Architecture

3.1 System Level Architecture



3.2 Software Architecture



4 Design Strategy

Our goal is to build a phishing detection system that is reliable and capable of providing accurate results. Our strategy is to design Deep Scan in two phases so we can complete our project in time, and build our FYP project that is completely working.

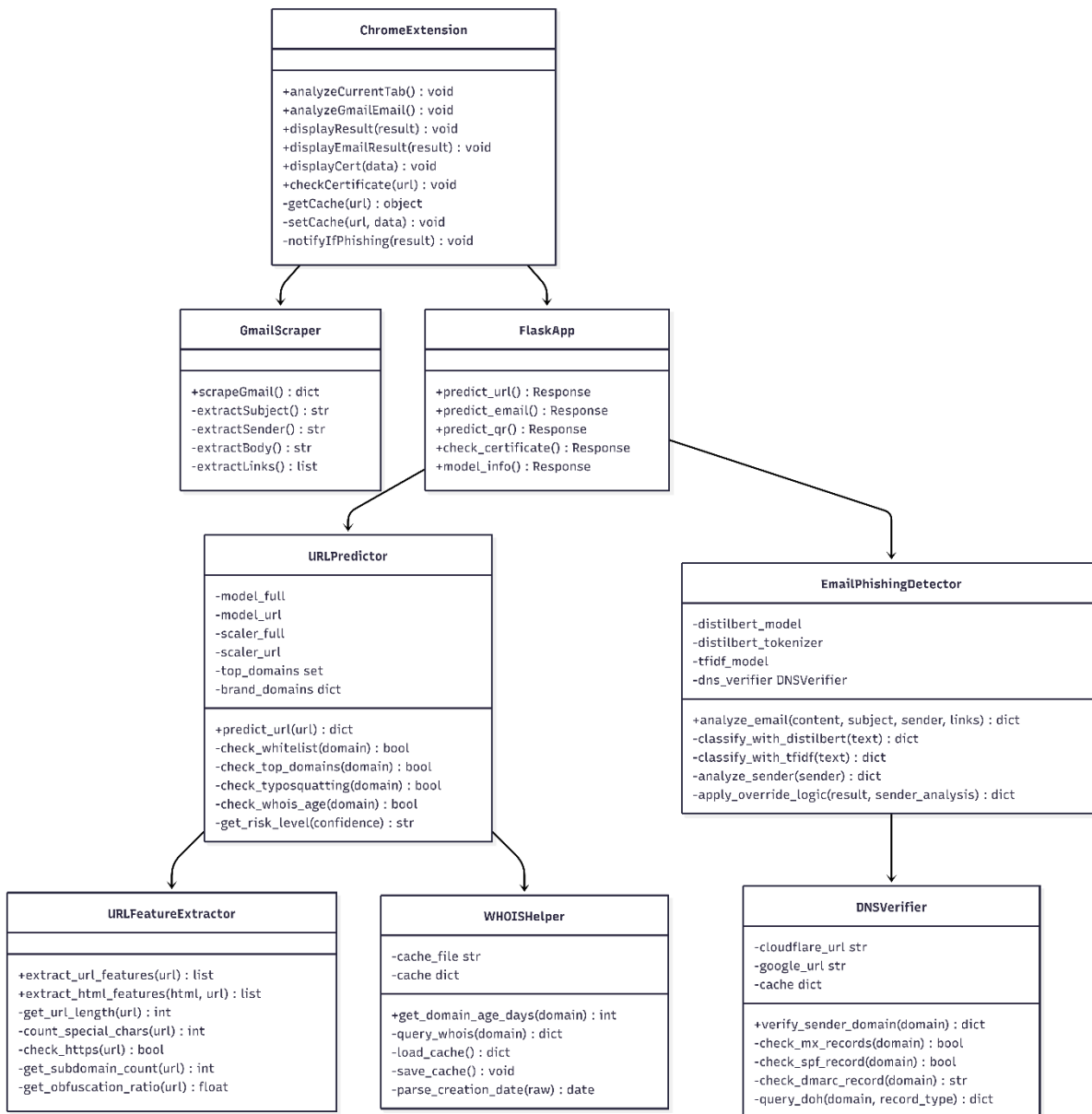
For URL analysis, the system first analyzes the webpage and perform analysis using both URL and HTML features. If the webpage features are not available to be analyzed because of internet issues, or other restrictions, then the system switches to URL only Random Forest model. For email analysis, the system first uses the DistilBERT model, but if the model cannot be loaded then the system uses TF-IDF model. By using this approach the system continues to classify both URL and emails without causing interruptions to the users.

For QR codes detection, we will extract URLs from the QR codes and then analyze them through our URL analysis module.

Since we aim to provide fast and accurate results to the users, the Chrome Extension stores recently analyzed results for a short period of time so that the websites that are scanned again, the system provides the stored result immediately.

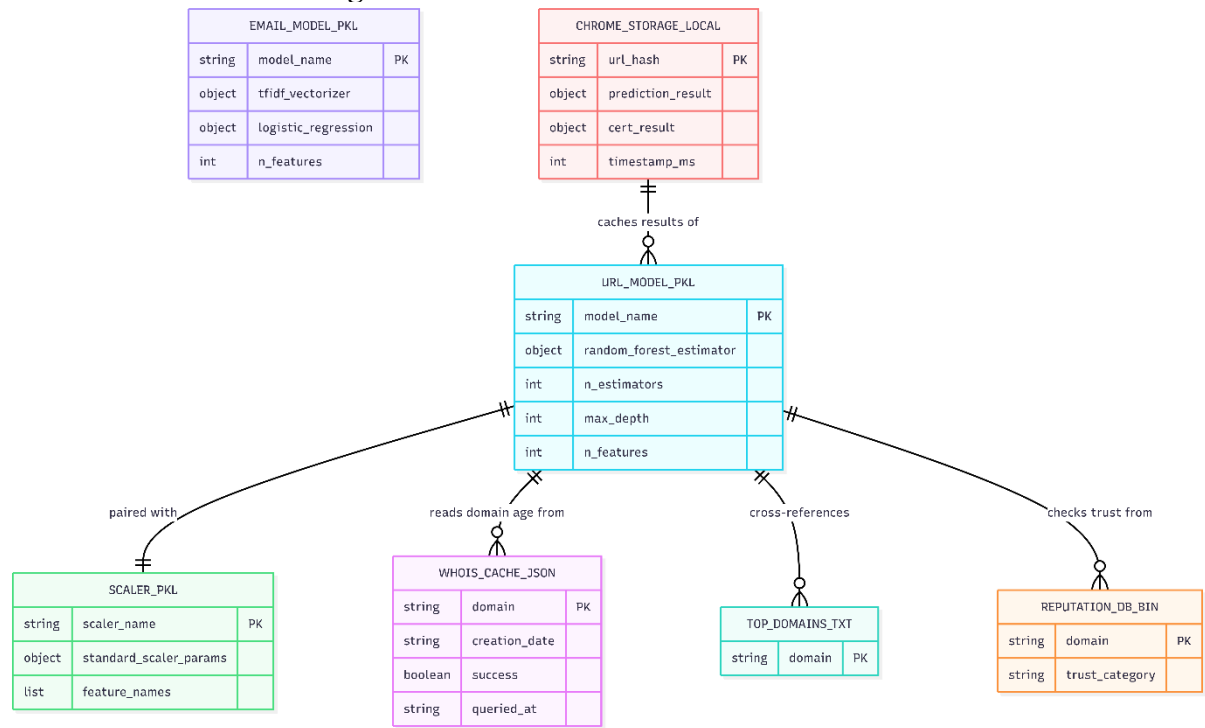
5 Detailed System Design

5.1 Design Class Diagram



5.2 Database Design

5.2.1 File Based Database Design:



5.2.2 Data Dictionary

5.2.2.1 Data 1

Data 1						
Name		URL Input				
Alias		Target URL, Input URL, Test URL				
Where-used/how-used		Input to prediction process via web interface and API endpoints				
Content description		String containing web URL for phishing analysis				
Field Name	Description	Type	Length	Null able	Default Value	Key Type
url	Web URL to be analyzed	String	2048	No	-	-

5.2.2.2 Data 2

Data 2						
Name		URL prediction Result				
Alias		Classification Output, Detection Result				
Where-used/how-used		Output from prediction process, returned to web interface				
Content description		JSON object containing prediction results and confidence scores				
Field Name	Description	Type	Length	Null able	Default Value	Key Type
Prediction	Classification result (Legitimate/Phishing)	String	20	No	-	-
Confidence	Prediction confidence score	Float	-	No	0.0	-
probabilities.phishing	Phishing probability score	Float	-	No	0.5	-
probabilities.legitmate	Legitmate probability score	Float	-	No	0.5	-
features_count	Number of features analyzed	Integer	-	No	50	-

5.2.2.3 Data 3:

Data 3						
Name	Feature Vector					
Alias	URL Features, Analysis Features, ML Features					
Where-used/how-used	Internal processing in feature extraction pipeline					
Content description	50-element numerical array representing URL characteristics					
Field Name	Description	Type	Length	Null able	Default Value	Key Type
URLLength	Length of the UR	Integer	-	No	0	-
IsHTTPS	Whether URL uses HTTPS (0/1)	Integer	1	No	0	-
HasTitle	Whether page has title (0/1)	Integer	1	No	1	-
Bank	Contains banking keywords (0/1)	Integer	1	No	0	-

5.2.2.4 Data 4:

Data 4						
Name	Email Input					
Alias	Email Content					
Where-used/how-used	Input to email prediction process via web interface and API					
Content description	JSON object containing email data for phishing analysis					
Field Name	Description	Type	Length	Null able	Default Value	Key Type
Content	Email body text	String	1000	No	-	-
Subject	Email subject line	String	200	Yes	""	-
Sender	Sender email address	String	254	No	-	-

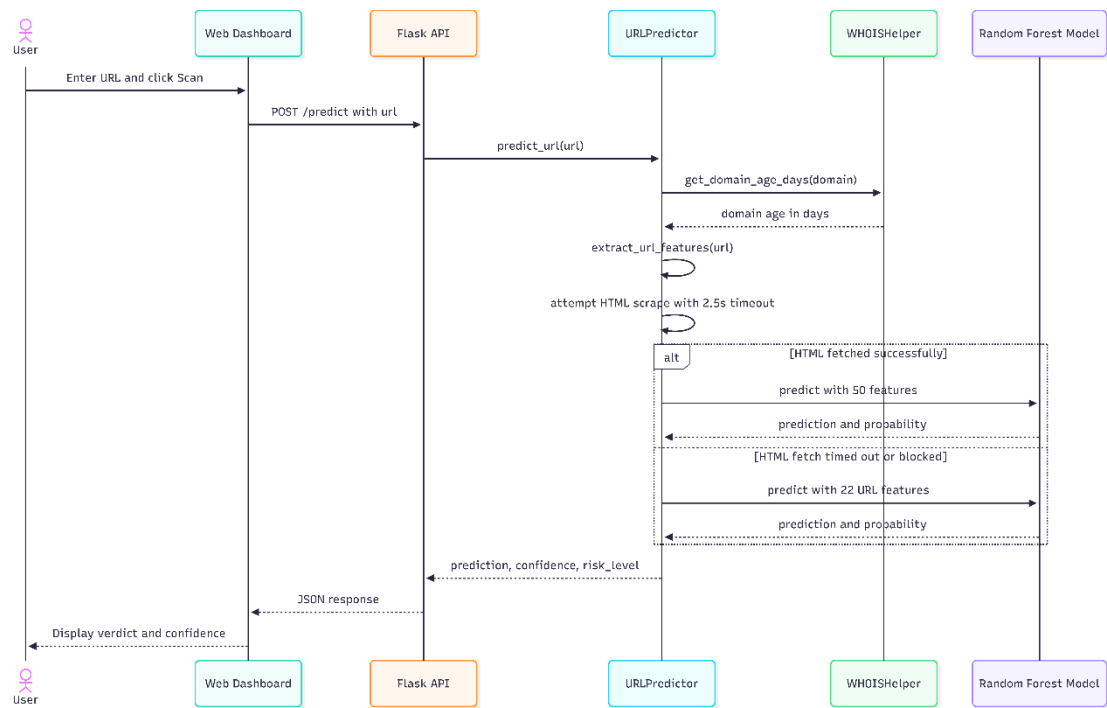
5.2.2.5 Data 5

Data 5						
Name	Email prediction Result					
Alias	Email Classification Output, Email Detection Result					
Where-used/how-used	Output from email prediction process					
Content description	JSON object containing email analysis results					
Field Name	Description	Type	Length	Null able	Default Value	Key Type
Prediction	Email classification (Legitimate/Phishing)	String	20	No	-	-
confidence	Email prediction confidence	Float	-	No	0.0	-
Risk_level	Risk assessment (Low/Medium/High)	String	10	No	"Low"	-
Method	Detection method used	String	20	No	NLP	-

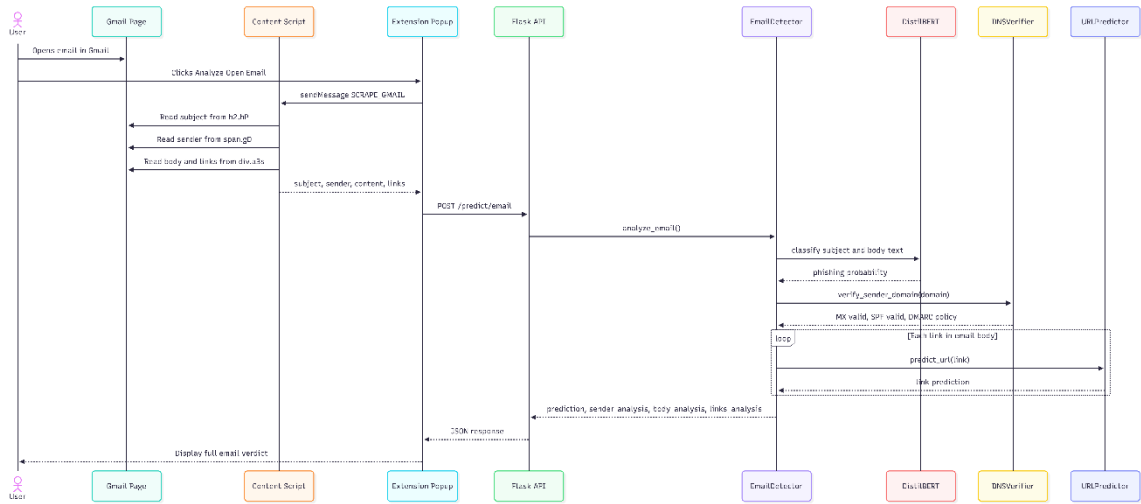
5.3 Application Design

5.3.1 Sequence Diagram

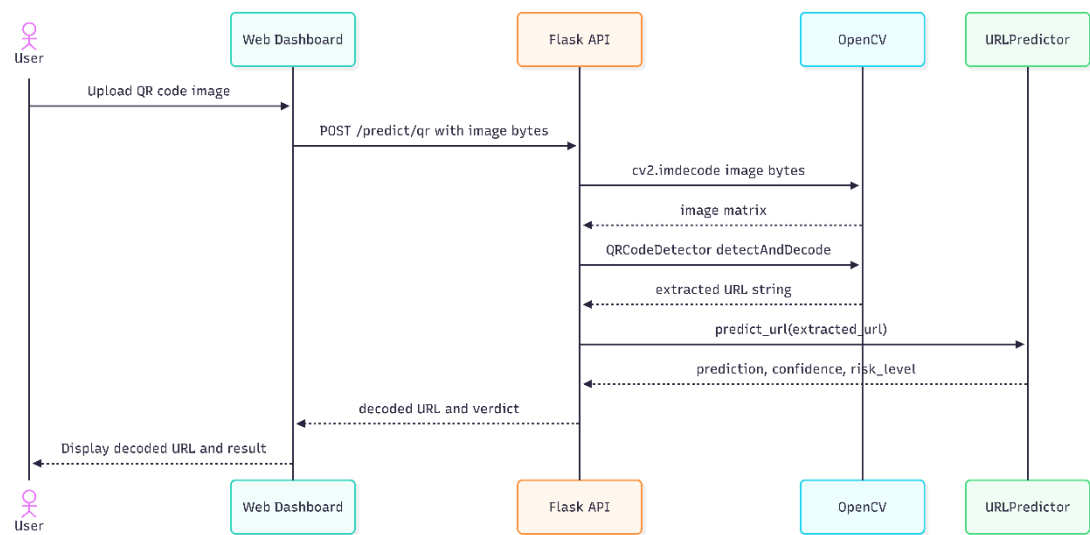
URL Scan(Web Dashboard)



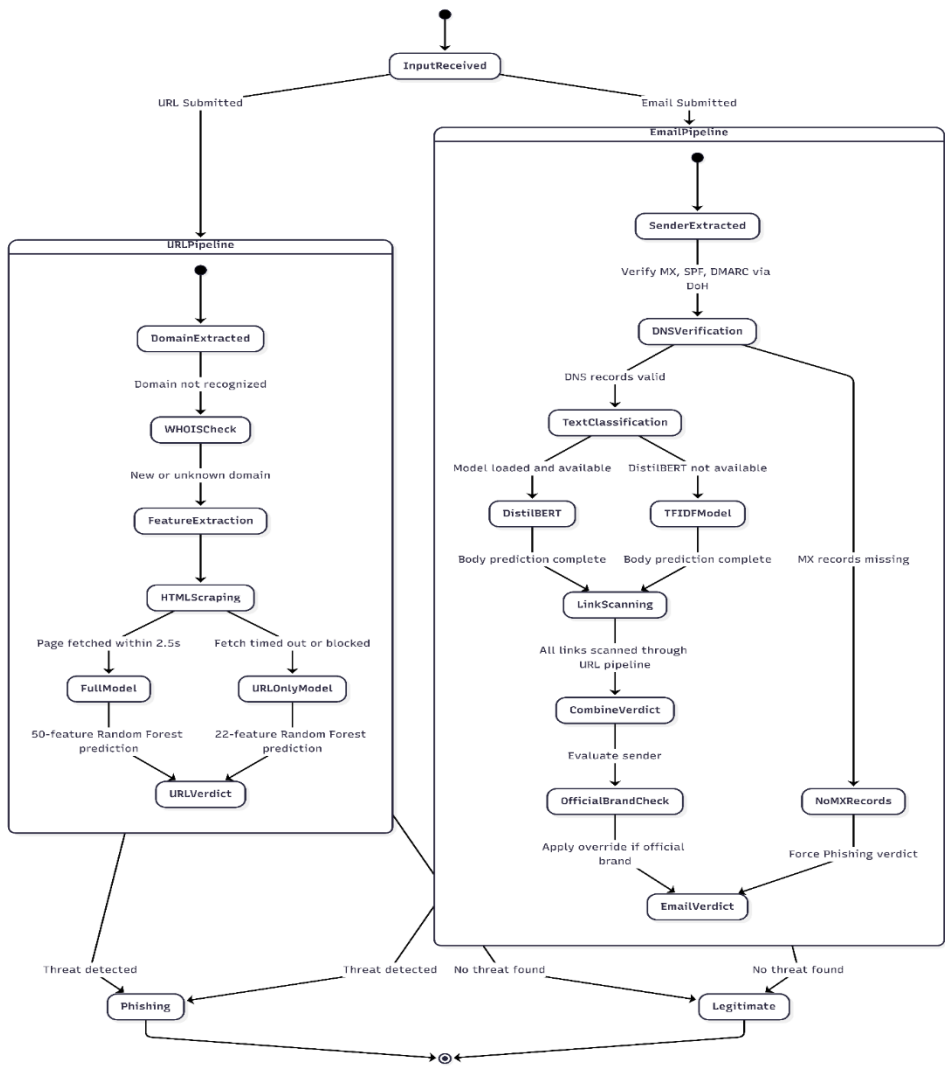
Email Scan(Gmail Extension)



QR Code Scan



5.3.2 State Diagram



1. Kalla, D., & Kuraku, S. (2023). *Phishing website URL's detection using NLP and machine learning techniques*. Journal on Artificial Intelligence, 5(1), 145–162.
<https://doi.org/10.32604/jai.2023.043366>
2. Lingaiah, G., & Pandey, R. K. (2023). *Comparative performance study of machine learning algorithms for phishing URL detection in cyber security systems*. Journal of Computational Analysis and Applications, 31(4), 1865–1873.
3. Sahingoz, O. K., Buber, E., Demir, O., & Diri, B. (2019). Machine learning based phishing detection from URLs. Expert Systems with Applications , 117, 345- 357.
<https://doi.org/10.1016/j.eswa.2018.09.029>
4. Tsai, E., Kumar, D., Raman, R. S., Li, G., Eiger, Y., & Ensafi, R. (2023). *CERTainty: Detecting DNS manipulation at scale using TLS/SSL certificates*. Proceedings on Privacy Enhancing Technologies, 2023(3), 122–137. <https://doi.org/10.56553/popets-2023-0073>

A4. OTHER TECHNICAL DETAIL DOCUMENTS

Test Cases Document

Software Test Plan

The following test plan covers all the screens and modules involved in Deep Scan system

S. No	Description	Test Engineer	Start Date	End Date
1	URL Analysis (Screen 1)	Mir Balach	Week 3	Week 4
2	Email Analysis (Screen 2)	Mir Balach	Week 4	Week 5
3	Chrome Extension (Screen 3)	Mir Balach	Week 6	Week 6

TC-01 — URL Analysis (Screen 1)

Project Name: Deep Scan

Iteration No: FYP-II

Module Name: Email Analysis

Date: Week 4

Test Case ID: TC-01

Test Engineer: Mir Balach

Test Case Description: Verify the working of URL Analysis. Test empty submission, secure submission, phishing submission

S. No	Steps	Input Data	Expected Result	Actual Result	Pass/Fail
TC-01-1	Empty submission in URL search box	NIL	Error	Error	Pass
TC-01-2	Enter legitimate URL for analysis Press Scan button	www.google.com	URL is secure	URL is secure	Pass
TC-01-3	Enter phishing URL for analysis Press Scan button	www.g00gle.com	URL is suspicious	URL is suspicious	Pass

TC-02 — Email Analysis (Screen 2)

Project Name: Deep Scan

Iteration No: FYP-II

Module Name: Email Analysis

Date: Week 5

Test Case ID: TC-02

Test Engineer: Mir Balach

Test Case Description: Verify the working of Email Analysis. Test empty submission, secure email submission, phishing email submission

S. No	Steps	Input Data	Expected Result	Actual Result	Pass/Fail
TC-02-1	Empty submission in Email analysis box	NIL	Error	Error	Pass
TC-02-2	Enter legitimate Email for analysis Press Scan button	"Hi Abdullah, How are you"	Email is secure	Email is secure	Pass
TC-02-3	Enter phishing Email for analysis Press Scan button	"Dear Customer, Your bank account has been blocked. Please click the link below to verify your account and continue using online banking. Thank you. "	Email is suspicious	Email is suspicious	Pass

TC-03 – Chrome Extension (Screen 3)

Project Name: Deep Scan

Iteration No: FYP-II

Module Name: Chrome Extension

Date: Week 6

Test Case ID: TC-03

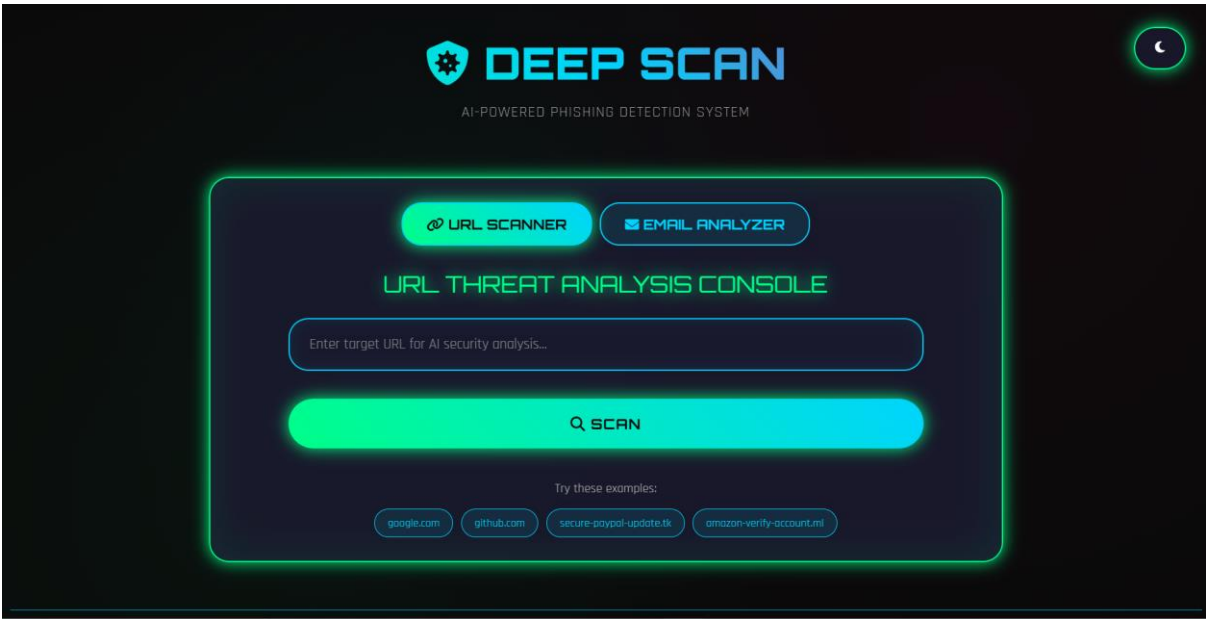
Test Engineer: Mir Balach

Test Case Description: Verify the working of Chrome Extension

S. No	Steps	Input Data	Expected Result	Actual Result	Pass/Fail
TC-03-1	Click on Extension shortcut to see analysis result while the system is offline	NIL	Error	Error	Pass
TC-03-2	Click on Extension shortcut to see analysis result on legitimate website while the system is online	Open www.google.com and click on the extension and then click on scan.	Secure	Secure	Pass
TC-03-3	Click on Extension shortcut to see analysis result on suspicious website while the system is online	Open http://paypal.example.com-login.verify-account.example.com/login and click on the extension and then click on scan.	Suspicious	Suspicious	Pass

UI/UX Detail Document

1. URL Analysis Tab:



2. Email Analysis Tab

URL SCANNER

EMAIL ANALYZER

EMAIL NLP ANALYSIS CONSOLE

Sender email (optional)

Email subject (optional)

Paste email content here for NLP analysis...

SCAN

Try these examples:

Phishing Email

Legitimate Email

3. URL Analysis (Secure URL)

URL THREAT ANALYSIS CONSOLE

https://www.google.com

SCAN

Try these examples:

google.com

github.com

secure-paypal-update.tk

amazon-verify-account.ml

✓

SECURE

TARGET VERIFIED AS LEGITIMATE BY AI ANALYSIS

98%

SECURITY RATING

AI-Powered Analysis

📏 URL LENGTH

22 CHARS

🔒 ENCRYPTION

SECURED

⚠️ RISK LEVEL

VERY LOW

📊 CONFIDENCE

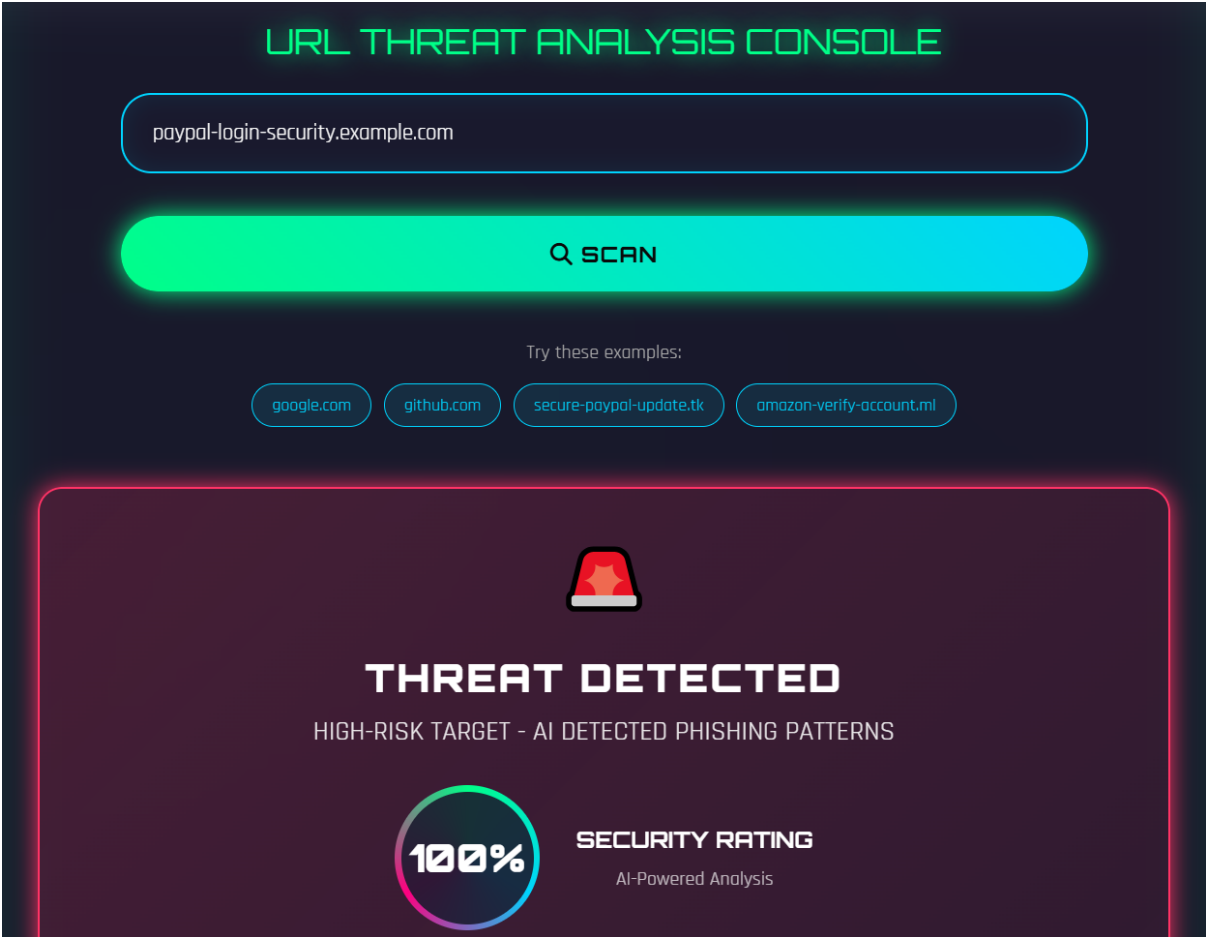
98%

🌐 SSL CERTIFICATE

✓ VALID

Expires: 2026-08-31 (59d) · Google Trust Services

4. URL Analysis(Suspicious URL)



5. Email Analysis (Secure Email)

Hello Team,

This is a reminder that our meeting will be held tomorrow at 10 AM in the conference room.

Thank you.

Q SCAN

Try these examples:

Phishing Email

Legitimate Email

✓

SECURE

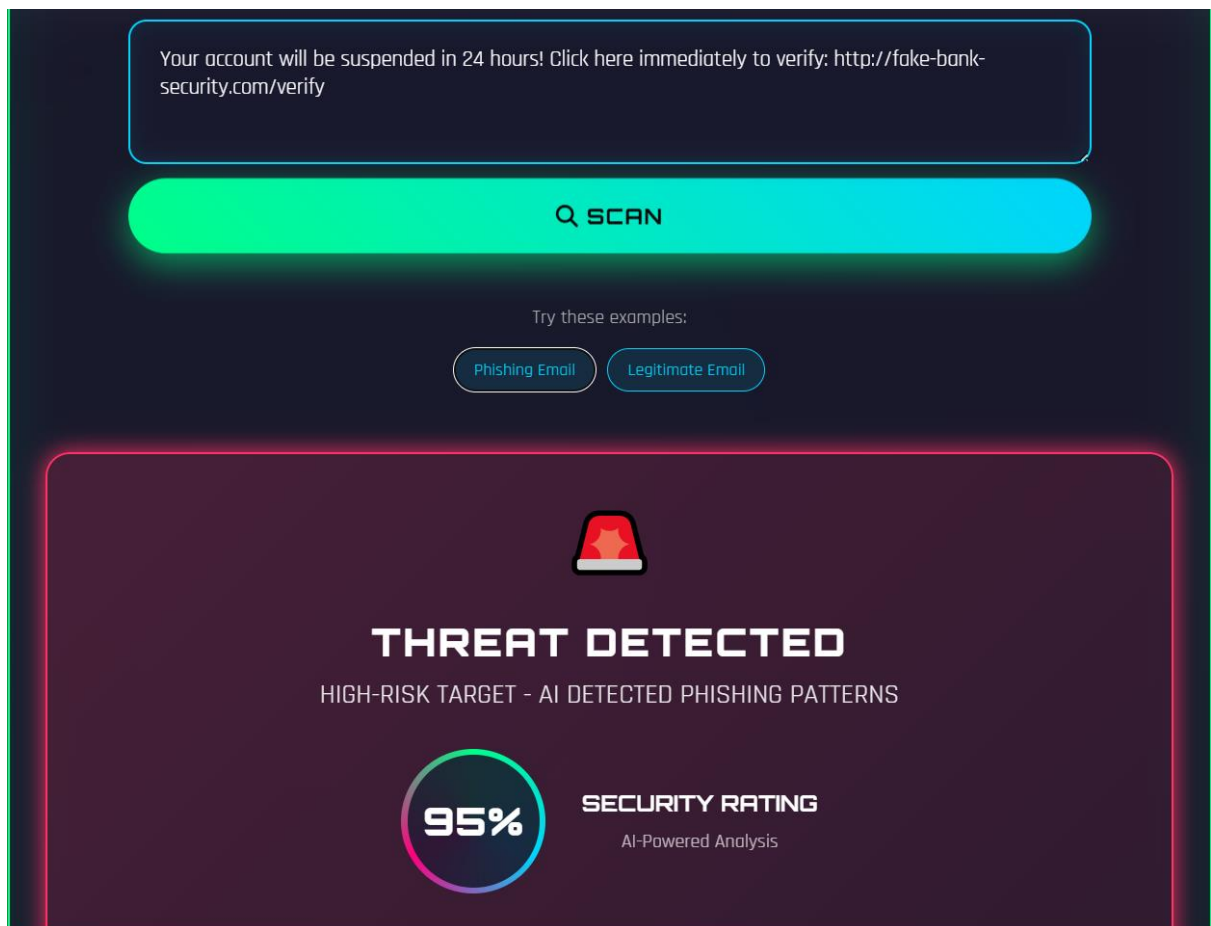
TARGET VERIFIED AS LEGITIMATE BY AI ANALYSIS

92%

SECURITY RATING

AI-Powered Analysis

6. Email Analysis (Suspicious Email)



7. Chrome Extension



DEEP SCAN

AI-Powered Phishing Detection

 <https://chatgpt.com/c/6a34a05e-2074-83ee-a22...>



SECURE

Site appears legitimate. (89% safe)



CONFIDENCE

89%



RISK LEVEL

Very Low



PHISHING PROB

0%



SAFE PROB

100%



SSL CERTIFICATE

 **VALID**

Expires: 2026-08-07 (35d) · Google Trust Services

 **Recheck**

 **Dashboard**

Deep Scan v2.0 | ML + NLP Detection

Coding Standards Document

```
190 @app.route(rule='/check_certificate', methods=['POST'])
191 def check_certificate():
192     try:
193         data = request.get_json()
194         if not data or 'url' not in data:
195             return jsonify({'error': 'Missing URL'}), 400
196
197         url = data['url'].strip()
198         if not url.startswith(('http://', 'https://')):
199             url = 'https://' + url
200
201         parsed = urlparse(url)
202         hostname = parsed.netloc or parsed.path
203         hostname = hostname.split(':')[0] # remove port if any
204
205         # Check if it's even HTTPS
206         if not url.startswith('https://'):
207             return jsonify({
208                 'has_certificate': False,
209                 'is_valid': False,
210                 'status': 'No Certificate',
211                 'message': 'Site uses HTTP, no SSL/TLS certificate'
212             })
213
214         try:
215             ctx = ssl.create_default_context()
216             with ctx.wrap_socket(socket.create_connection(address=(hostname, 443), timeout=5), server_hostname=hostname) as s:
217                 cert = s.getpeercert()
218
219             # Parse expiry date
220             expire_str = cert.get('notAfter', '')
221             expire_date = datetime.strptime(expire_str, format='%b %d %H:%M:%S %Y %Z')
222             is_expired = expire_date < datetime.utcnow()
223             days_left = (expire_date - datetime.utcnow()).days
```

```

subject = dict(x[0] for x in cert.get('subject', []))
issuer = dict(x[0] for x in cert.get('issuer', []))

return jsonify({
    'has_certificate': True,
    'is_valid': not is_expired,
    'status': 'Expired' if is_expired else 'Valid',
    'expires': expire_date.strftime('%Y-%m-%d'),
    'days_remaining': days_left,
    'issued_to': subject.get('commonName', hostname),
    'issued_by': issuer.get('organizationName', 'Unknown')
})

except ssl.SSLCertVerificationError:
    return jsonify({
        'has_certificate': True,
        'is_valid': False,
        'status': 'Invalid',
        'message': 'Certificate verification failed (untrusted/self-signed)'
    })

except (socket.timeout, ConnectionRefusedError, socket.gaierror):
    return jsonify({
        'has_certificate': False,
        'is_valid': False,
        'status': 'Unreachable',
        'message': 'Could not connect to host'
    })

except Exception as e:
    logger.error(f"Certificate check error: {str(e)}")
    return jsonify({'error': 'Failed to check certificate'}), 500

```

Project Policy Document

1. Purpose

The purpose of this project is to detect phishing URLs and phishing emails using Machine Learning and Natural Language Processing (NLP). The system is developed for educational and research purposes to improve user awareness and online security.

2. Scope

- The system analyzes URLs, QR codes and emails for phishing detection.
- It checks the SSL certificate of websites.
- It provides results through a web application and a Chrome extension.

3. User Responsibilities

- Users should enter valid URLs and email content for analysis.
- Users should review the system's results before taking any action.

4. Data Privacy

- The system does not store users' URLs or email content permanently.
- User data is used only for analysis.

5. Security Policy

- The system checks website SSL certificates to improve security.
- Regular updates should be made to improve detection against new phishing attacks.

6. Limitations

- The system may not detect every phishing attack.
- New phishing techniques may require retraining of the machine learning models.
- Email analysis currently supports only English language emails.

7. Maintenance Policy

- The datasets and machine learning models should be updated regularly.
- Bugs should be fixed as they are found.
- New features can be added in future versions.

8. Ethical Policy

- The project is developed only for educational purposes.
- It must not be used to create, support, or promote phishing attacks.
- Users should use the system responsibly and legally.

9. Conclusion

This project follows basic security, privacy, and ethical guidelines to provide a safe and reliable phishing detection system. Future improvements will help increase its accuracy and performance.

User Manual Document

1. Introduction

Deep Scan is an AI-powered phishing detection system that helps you identify malicious URLs and phishing emails. It uses machine learning and real-time threat intelligence to protect you from online threats.

2. Installation & Setup

Step 1 — Install Dependencies

Open a terminal in the project root folder and run:

```
pip install -r backend/requirements.txt
```

Step 2 — Start the Backend Server

```
python backend/app.py
```

You should see:

API will be available at: <http://localhost:5000>

Step 3 — Open the Web Interface

Open frontend/cyberinterface.html in your browser. No additional server needed.

Step 4 — Load the Chrome Extension (Optional)

1. Open Chrome and go to `chrome://extensions/`
2. Enable **Developer Mode** (top right toggle)
3. Click **Load unpacked**
4. Select the `chrome_extension/` folder
5. The Deep Scan icon will appear in your toolbar

4. Using the Web Interface

4.1 Scanning a URL

1. Open frontend/cyberinterface.html in your browser
2. Type or paste a URL into the input field (e.g. https://example.com)
3. Click SCAN
4. Wait for the result

Result panel shows:

- **Threat Status:** SECURE or THREAT DETECTED
- **Security Rating:** Confidence percentage
- **URL Length:** Character count of the URL
- **Encryption:** Whether HTTPS is used
- **Risk Level:** Very Low / Low / Medium / High
- **Confidence:** How certain the system is
- **SSL Certificate:** Certificate validity and expiry

4.2 Quick Test Examples

The interface provides 4 built-in test buttons:

Button	Expected Result
Google.com	Legitimate
Github.com	Legitimate
secure-paypal-update.tk	Phishing
amazon-verify-account.ml	Phishing

4.3 Email Analysis

1. Navigate to the email analysis section
2. Paste the email body into the **Content** field
3. Optionally fill in **Subject** and **Sender** fields
4. Click **Scan**
5. The system returns a phishing probability and risk score

5. Using the Chrome Extension

1. Click the extension icon in your Chrome toolbar
2. The extension shows the risk level of the current page's URL
3. A **notification** will appear if a high-risk URL is detected while browsing

A5. FLYER & POSTER DESIGN

A6. COPY OF EVALUATION COMMENTS

COPY OF EVALUATION COMMENTS BY SUPERVISOR FOR PROJECT – I MID SEMESTER EVALUATION

Mr. Wajahat	P23F25 - Deep Scan	Good progress
-------------	--------------------	---------------

**COPY OF EVALUATION COMMENTS BY SUPERVISOR FOR
PROJECT – I END SEMESTER EVALUATION**

COPY OF EVALUATION COMMENTS BY JURY FOR PROJECT – I
END SEMESTER EVALUATION

23	P23F25 - Deep Scan	Revise the scope
----	--------------------	------------------

--	--	--

23	P23F25 - Deep Scan	Revision of scope is advised
23	P23F25 - Deep Scan	Revise the scope.
23	P23F25 - Deep Scan	No proper work flow and clear USP, there is need to be improvement

**COPY OF EVALUATION COMMENTS BY SUPERVISOR FOR
PROJECT – II MID SEMESTER EVALUATION**

**COPY OF EVALUATION COMMENTS BY SUPERVISOR FOR
PROJECT – II END SEMESTER EVALUATION**

A7. MEETINGS' MINUTES

A Photostat or scanned copy should be placed when submitting document to Project Coordinator.

Document should be signed by the supervisor and all other members present in the meeting (wherever possible). (**Note:** Please remove this line when attach copy that is required)

Weekly meetings' minutes are required (held with Supervisor and/or with client). Important group discussions can also be included here.

A8. DOCUMENT CHANGE RECORD

Date	Version	Author	Change Details
30/June/2026	1.0	All Members	

A9. PROJECT PROGRESS

E29								
	A	B	C	E	F	G	H	I
1	[P23F25] Deep Scan							
2	FYP - Fall 25							
3								
4	Project Start Date: 11/30/2025 (Sunday) y Week 7							
5	Team Members: Mir Balach Kai Ghulam Mustafi Laksh P Abdul Wahab In							
6								
7	WBS	TASK	Assigned To	START	DAYS	END	% DONE	WORK DAYS
8	1	FYP-I				-		-
9	.1	Requirements Analysis & All Members		Sun 11/30/25	14	Sat 12/13/25	100%	10
10	.1	Data Collection & Cleanir	Mir Balach	Sun 12/14/25	21	Sat 1/03/26	100%	15
11	.1	URL Model Training	Mir Balach	Sun 12/14/25	28	Sat 1/10/26	100%	20
12	.1	Email Rule-based Engine	Ghulam Mustafa	Sun 1/04/26	21	Sat 1/24/26	100%	15
13	.1	Frontend	Laksh & Wahab	Sun 1/04/26	14	Sat 1/17/26	100%	10
14	.1	Flask API Integration	Balach & Musta	Sun 1/11/26	15	Sun 1/25/26	100%	10
15	.1	Initial Testing & Documentation	All Members	Mon 1/12/26	8	Mon 1/19/26	100%	6
16	1	FYP-II				-		-
17	.1	Research Advanced ML	Mir Balach	Sun 2/22/26	21	Sat 3/14/26	100%	15
18	.1	Chrome Extension Devel	Laksh & Wahab	Sun 2/22/26	35	Sat 3/28/26	100%	25
19	.1	Advance Email Analysis i	Balach & Musta	Sun 3/29/26	28	Sat 4/25/26	100%	20
20	.1	System Integration and h	Ghulam Mustafa	Sun 4/26/26	21	Sat 5/16/26	100%	15
21	.1	Performance Tuning & Fi	All Members	Sun 5/17/26	14	Sat 5/30/26	100%	10
22	.1	Final Documentation	All Members	Sun 5/31/26	21	Sat 6/20/26	100%	15
23	.1	Final Presentation Prep &	All Members	Sun 6/21/26	21	Sat 7/11/26	100%	15

A10. RESEARCH PAPER

Deep Scan

Mir Balach Kamal
BSCS

DHA Suffa University Karachi
Karachi, Pakistan

Abdul Wahab Imran
BSCS

DHA Suffa University Karachi
Karachi, Pakistan

Laksh Prince

DHA Suffa University Karachi
Karachi, Pakistan

Ghulam Mustafa
BSCS

DHA Suffa University Karachi
Karachi, Pakistan

Abstract— Phishing attacks remain one of the top threats to internet users and organizations every year. Attackers keep finding new ways to steal information, including fake websites, fake emails, and now even fake QR codes. This paper presents **Deep_Scan**, a phishing detection system that works on three types of threats at the same time: malicious URLs, phishing emails, and QR code-based attacks (called "quishing"). The system uses a three-phase approach for URL checking, which goes from fast rule-based checks to machine learning models trained on the PhiUSIIL dataset (235,795 URLs). For email analysis, it uses a DistilBERT transformer model as the primary detector, backed by a TF-IDF logistic regression model, and a rule-based fallback. It also checks sender domain authenticity through real-time DNS verification of MX, SPF, and DMARC records. QR code images are decoded using OpenCV, and the extracted URL is then analyzed by the same URL pipeline. Domain WHOIS age checks and Levenshtein distance typosquatting detection further reduce false negatives. Testing shows the URL model achieves around 95% accuracy on the PhiUSIIL dataset, and the email model achieves strong performance on the CEAS_08 and Enron combined corpus. A Google Chrome extension provides real-time protection in the browser. All components are connected through a Flask REST API backend.

Keywords— *phishing detection, machine learning, DistilBERT, Random Forest, QR code analysis, DNS verification, typosquatting, WHOIS, Chrome extension*

Introduction

The internet has become central to daily life. People use it for banking, shopping, communication, and work. But this also makes it a target for criminals who want to steal personal data, passwords, or money. One of the most common methods attackers use is phishing — creating fake versions of real websites or emails to trick users into giving away their information.

According to various reports, billions of phishing attempts happen every year. No single detection method works against all of them. Older systems that only check URLs are fooled by attackers who move to email campaigns. Email-only systems

miss dangerous links embedded in QR codes. This fragmented protection leaves users exposed.

Deep Scan was built to solve this problem by creating a single unified system that can handle three types of phishing attacks:

1. **URL phishing** — detecting dangerous websites before the user visits them
2. **Email phishing** — analyzing email content, sender authenticity, and embedded links
3. **QR code phishing (quishing)** — scanning QR codes to find hidden malicious URLs

This paper describes how the system works, the technologies used, the datasets, the evaluation results, and the design decisions made throughout development.

Background and Related Work

Early phishing URL detection relied on blacklists and simple keyword rules. Over time, researchers moved toward machine learning. Studies showed that Random Forest classifiers work well for URL classification because they handle many different feature types at once and resist overfitting [1]. The PhiUSIIL Phishing URL Dataset is a modern, large-scale benchmark with over 235,000 labeled URLs and both structural URL features and web content features.

Traditional email spam filters use keyword matching and sender blacklists. More recent work applies NLP techniques. Research using BERT-based models on email corpora showed better accuracy than TF-IDF or bag-of-words methods [2]. The CEAS_08 dataset is a well-known benchmark for phishing email classification. The Enron email corpus is a large public collection of legitimate corporate emails.

QR code usage grew sharply after 2020. Attackers now embed malicious URLs inside QR codes on posters, in emails, or in messages. This is called "quishing." Few existing systems include QR code scanning as part of their phishing detection pipeline.

URL Phishing Detection

When the system can successfully fetch the webpage HTML (within a 2.5-second timeout), it uses a full 50-feature model combining:

22 URL-structural features computed only from the URL text, 28 web content features extracted from the live HTML page

The 22 URL features include: URL length, domain length, whether the domain is an IP address, TLD legitimacy probability, character continuation rate, alphabetic character probability, number of subdomains, obfuscation detection (percent-encoded characters, long random strings), digit/letter ratios, counts of '=', '?', '&', special characters, and HTTPS usage.

The 28 HTML features include: lines of code, largest line length, page title presence, title-domain similarity score, favicon presence, viewport responsiveness, number of HTTP redirects, self-referencing links, meta description, popup scripts, iFrames, external form submissions, social network links, submit buttons, hidden form fields, password input fields, banking/payment/crypto keywords in page text, copyright notice, image count, CSS file count, JavaScript count, and counts of self/empty/external anchor links.

URL-Only 22-Feature Model

If the webpage cannot be fetched (offline, blocks bots, or times out), the system falls back to a URL-only 22-feature model that works purely on the URL text itself. This is always available.

If both ML models are unavailable (e.g., model files not found), a heuristic scoring fallback runs. It evaluates the URL based on HTTPS usage, domain segment count, TLD quality, URL length, and special character count.

Email Phishing Detection

Email subject and body are concatenated, tokenized to a maximum of 512 tokens, and passed through the model. It produces four probability outputs: 'legitimate_email', 'phishing_url', 'legitimate_url', and 'phishing_url_alt'. The final phishing probability is the sum of the two phishing label probabilities.

A hybrid speedup mode is available: TF-IDF runs first, and if it gives a high-confidence legitimate result (≥ 0.90), DistilBERT is skipped. This reduces inference time for clearly legitimate emails.

QR Code Detection

QR code usage grew rapidly after 2020. Attackers began hiding phishing URLs inside QR codes placed on printed materials, emails, or social media. A user scanning a malicious QR code may be redirected to a phishing site without ever seeing the URL. Existing phishing tools do not address this threat.

URLs in QR codes are extracted through OpenCV and checked through URL detection module and results are displayed to the user.

SSL Certificate Verification

An additional endpoint checks the SSL certificate of a given domain. Python's 'ssl' library opens a real TLS connection on port 443, reads the certificate, and returns:

- Whether the site uses HTTPS at all (HTTP sites have no certificate)
- Whether the certificate is valid or expired
- Days remaining until expiry
- Who the certificate was issued to (common name)
- Which certificate authority issued it

This helps users verify that a site has a properly trusted certificate, since phishing sites often use self-signed certificates or recently issued certificates from free CAs.

Chrome Extension

The system allows to check URLs and emails through chrome extension, users can check emails by opening the emails in their Gmail inbox and click on extension icon to check if the email is secure or suspicious.

Conclusion

This paper presented Deep_Scan, a multi-modal phishing detection system that goes beyond single-threat approaches. By combining three-phase URL detection, multi-tier email analysis using DistilBERT and TF-IDF, real-time DNS sender verification, WHOIS domain age checks, Levenshtein typosquatting detection, and QR code quishing analysis — all connected through a Flask REST API — the system provides a broad defense against modern phishing attacks.

The URL model achieves approximately 95–96% accuracy on the PhiUSIIL benchmark. The email system's multi-tier fallback ensures reliable detection even when heavyweight models are unavailable. The QR code module extends coverage to a growing and underserved threat vector.

The core finding is that combining fast deterministic rules with trained machine learning models is more effective than either alone. Rule-based checks catch known brand impersonation with certainty. ML models generalize to unseen patterns. DNS verification adds a ground-truth signal that no text model can replicate. Together, these layers reduce both false positives and false negatives across all three threat types.

References

- [1] A. Khade and S. Shah, "Phishing website detection using machine learning algorithms," *International Journal of Computer Applications*, vol. 182, no. 34, 2019.
- [2] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019.
- [3] S. Hannousse and S. Yahiouche, "PhiUSIIL phishing URL dataset," UCI Machine Learning Repository, 2023. [Online]. Available: <https://www.kaggle.com/datasets/harisudhan411/phishing-and-legitimate-urls>
- [4] CEAS 2008 Email Spam Filtering Challenge Dataset. [Online]. Available: <https://plg.uwaterloo.ca/~gvcormac/ceascorpus/>
- [5] Sahingoz, O. K., Buber, E., Demir, O., & Diri, B. (2019). Machine learning based phishing detection from URLs. *Expert Systems with*

Applications, 117, 345-357.
<https://doi.org/10.1016/j.eswa.2018.09.029>

- [6] . Lingaiah, G., & Pandey, R. K. (2023). *Comparative performance study of machine learning algorithms for phishing URL detection in cyber security systems*. Journal of Computational Analysis and Applications, 31(4), 1865–1873.

